

Algo Que Pedir



Entrega 2

En esta entrega avanzaremos en nuevas iteraciones de requerimientos anteriores y en otros surgidos de nuevos relevamientos.

Delivery

Queremos poder darles la posibilidad de tener más de una condición de las ya conocidas, para permitir configurar por ejemplo "rango seguro ó pedido mayor a \$ 30.000" y la opción "pedidos certificados y locales determinados".

Cupón

Como estrategia de marketing nos pidieron poder aplicar cupones de descuentos a los pedidos. Estos cupones corren por parte de la plataforma, por lo que se terminan aplicando sobre el valor a pagar de los pedidos.

Los cupones funcionan de acuerdo a las siguientes características:

- **Fecha de emisión y duración:** Cada cupón tiene una fecha de emisión y un número determinado de días durante los cuales se mantiene válido.
- **Porcentaje base de descuento:** Cada cupón tiene un porcentaje de descuento base, que de ser un cupón aplicable, asegura como mínimo ese descuento.
- **Aplicabilidad a los pedidos:** El cupón debe cumplir con ciertos criterios para ser aplicable a un pedido.
 - No debe estar vencido.
 - No debe estar aplicado.
 - Debe asegurarse que al aplicarse el descuento, no sea mayor al monto a pagar original.
 - Cada cupón exige una condición para poder ser aplicado
- **Descuento especial:** Además del porcentaje base, dependiendo del cupón y ciertas circunstancias, se puede llegar a ampliar el descuento a aplicar. Por el momento se

lanzan los primeros cupones de descuentos, pero puede que a futuro se agreguen nuevos.

- Existen cupones que se pueden aplicar en días específicos (cada uno de ellos determina el día). Estos, si el pedido incluye algún plato que haya sido lanzado en un día de la semana que coincide con el día indicado por el cupón, tendrán un descuento adicional del 10%. En caso contrario, el descuento adicional será del 5%. (Ejemplo: pedido total a pagar \$ 1000 cupón porcentaje descuento base 30% \$ 300, si el cupón es aplicable para los días miércoles y algún plato del pedido fue lanzado un miércoles, el descuento es del 40% \$ 400, sino del 35% \$ 350)
- Otros que aplican el descuento si el pedido corresponde a alguno de los locales que se especifiquen en el cupón. En este caso el descuento especial no es un porcentaje, sino que se descuentan \$ 1000 si el pedido es certificado o \$ 500 si no lo es. (Ejemplo: si el pedido corresponde a alguno de los locales que detalla el cupón, y a su vez es certificado, el descuento adicional es de \$ 1000, pero si no lo es el descuento adicional es de \$ 500).
- Hay otros cupones que siempre pueden aplicarse y que además del porcentaje base, aplican un descuento especial por otro porcentaje hasta llegar a un tope de descuento máximo. (Ejemplo: cupón de descuento base 30%, descuento especial de 20% o tope de \$ 100, si el pedido es por valor a pagar de \$ 1000 el descuento especial sería de \$ 100. Si el pedido es por valor a pagar de \$ 400 el descuento especial sería de \$ 80.)

Pedido

Además de conocer de la entrega anterior el **total a pagar**, ahora se agrega el **total a pagar con cupón**.

Es importante para el negocio poder determinar si un pedido **tiene un cupón aplicado**, No hace falta saber cuál, solo saber si tiene. (Evitar comparaciones con tipo de cupones)

Usuario

Además de los tipos de usuarios que ya conocíamos, se agregan nuevos:

- Están los usuarios que con cumplir los requisitos generales les es suficiente para poder comer un plato.
- Hay otros que le gustaría poder combinar distintos tipos de los requisitos particulares que se ofrecen en la app, estos para poder comer un plato, deben cumplirse todos los requerimientos que quiera tener. Ejemplo si el usuario quiere ser **fiel e impaciente**, el plato debe estar elaborado por uno de sus locales preferidos y a su vez éste ser cercano.
- Hay usuarios que cambian según la edad, cuando tienen edad par se comportan como los **exquisitos**, sino son **conservadores**.

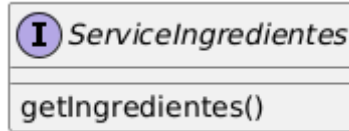
Repositorios

Para cada una de las entidades Usuario, Local, Delivery, Ingrediente, Plato, Cupón, se pide crear un Repositorio que tenga una colección de objetos en memoria, y que implemente el siguiente comportamiento:

- **void create(T object):** Agrega un nuevo objeto a la colección, y le asigna un identificador único (id). El identificador puede ser autoincremental para evitar que se repita.
- **void delete(T object):** Elimina el objeto de la colección.
- **void update(T object):** Modifica el objeto dentro de la colección. De no existir el objeto buscado, es decir, un objeto con ese id, se debe lanzar una excepción.
- **T getByid(Int id):** Retorna el objeto cuyo id sea el recibido como parámetro.
- **List<T> search(String value):** Devuelve los objetos que coincidan con la búsqueda de acuerdo a los siguientes criterios:
 - **Usuario:** El valor de búsqueda debe coincidir parcialmente con su nombre o apellido, o exactamente con su username.
 - **Local:** El valor de búsqueda debe coincidir parcialmente con el nombre o exactamente con el nombre de la calle.
 - **Delivery:** El valor de búsqueda debe coincidir con el comienzo del username.
 - **Ingrediente:** El valor de búsqueda debe coincidir con el nombre exacto.
 - **Plato:** El valor de búsqueda debe coincidir parcialmente con la descripción, o nombre, o nombre del local, o exactamente con el nombre de la calle del local.
 - **Cupón:** El valor de búsqueda debe corresponder exactamente con el porcentaje base.

Servicio de Actualización

Para mantener los datos de los ingredientes al día, nuestra plataforma debe ser capaz de comunicarse con un servicio externo y con los datos obtenidos actualizar o crear ingredientes nuevos en el Repositorio de Ingredientes.
Para ello nos proveen de esta interfaz.



Para actualizar nuestro repositorio de ingredientes utilizaremos el método **getIngredientes()** de ServiceIngredientes. El cual nos retornará un JSON¹ con el siguiente formato:

```
[
  {
    "id": 22,
    "nombre": "Leche",
    "costo": 200.5,
    "grupo": "LACTEOS",
    "origenAnimal": true
  },
  {
    "nombre": "Arroz",
    "costo": 100.5,
    "grupo": "CEREALES_Y_TUBERCULOS",
    "origenAnimal": false
  },
  {
    "id": 3,
    "nombre": "Azúcar",
    "costo": 200.0,
    "grupo": "AZUCARES_Y_DULCES",
    "origenAnimal": false
  },
  {
    "nombre": "Pollo",
    "costo": 400.5,
    "grupo": "PROTEINAS",
    "origenAnimal": true
  },
  {
    "id": 57,
    "nombre": "Manzana",
    "costo": 100.0,
    "grupo": "FRUTAS_Y_VERDURAS",
    "origenAnimal": false
  }
  (...)
]
```

¹ JavaScript Object Notation es un formato de intercambio de datos más liviano que el XML. Para más información ver <http://es.wikipedia.org/wiki/JSON> y <http://www.json.org/>. Parsers para JSON: [Jackson](#), [Kotlinx Serialization](#), [Gson](#), entre otros.

Se pide:

1. Diseñar e implementar el modelo de objetos de dominio.
2. Diseñar e implementar los casos de prueba correspondientes.
 - a. Armar el juego de datos necesario para realizar las pruebas.
 - b. Codificar los tests unitarios.
 - c. Usar mocks y/o stubs para testear los casos que impliquen llamadas al Services de Ingredientes
3. La entrega debe desarrollarse en un branch *develop*, que será mergeado en *master*, a través de un pull request, luego de ser aprobado por el tutor del grupo.
4. Integrar el proyecto con Github Action.
5. Agregar al README los badges del build y cobertura de test.
6. Generar un tag llamado “entrega-2” en el repositorio.